

- The command we will be using to manipulate the backend will be **mbpoll**.
 - It is a command line utility to read/write to ModBus addresses.
- The usage is as follows:
 - mbpoll -m tcp [IP of PLC container] -p 502 -a 0 -0 -t [register type] -r [ModBus address] [value to write to that address]
 - **To read instead of write**, simply omit the [value to write to that address] part of the command.
 - -t should be set to 0 for bools and 4 for ints.
 - “-a 0” and “-0” are fixed for the following reasons:
 - “-a 0” just specifies the slave device ID (which the user has no way of knowing, it’s just 0 in this case with the way I implemented it).
 - “-0” just tells the command to address from 0.
- The main goal of the challenge is to change 2 values in the EV charger’s **Inverter** PLC:
 - The **Is_Authenticated** bool variable to true (1)
 - The **Max_Voltage** int variable to any number $\geq$ the **Explosion_Threshold** variable.
 - You will not be able to see the names of these variables, only their values by polling their corresponding ModBus addresses via mbpoll.
 - You will need to logically intuit which addresses/values correspond to these key variables and change them accordingly.
- That covers the basics, but more detailed hints will be on the next page:

- **Finding the IP of the PLC container:**
 - Use the nmap command.
 - Simply type “nmap” for usage details/examples.
- **Finding the correct addresses to modify:**
 - Address range: %QX10.0-%QX10.1 and %QW20-%QW26.
 - %QX -> **bool** addresses, %QW -> **int** addresses.
 - Note that %QX registers are at a bit-level, meaning that we should multiply the integer part of an address by 8 when referencing it in mbpoll as a ModBus address (ex: %QX10.0 -> ModBus address 80 [10 x 8 + 0], %QX10.1 -> ModBus address 81 [10 x 8 + 1]).
 - Read the values from the address ranges to determine which ones you need to modify. You will need to logically intuit which addresses correspond to which variable (the important ones noted above) based on their values.
- **Resetting the backend:**
 - If you change variables incorrectly and mess up the backend, you can reset it by going onto the VM/machine that hosts the backend and typing the following commands:
 - `sudo docker compose stop`
 - `sudo docker compose up -d`
 - This will restart the containers. You will also need to go into either the pause menu or the main menu and re-enter the VM's IP so the frontend can reconnect to the backend.